NAME:- HITEN PAHILAJANI

ROLL NO:- 2401069

BATCH:- T13

SUBJECT:- ASD&D

# Experiment No. – 01

**Aim : To perform version control operations using Git**

**This experiment is mapped to LO2**

**LO2 : Use git effectively for version control source code managements and collaborative managements**

**Theory :**

Git is a distributed version control system used to track changes in source code and other files during software development. It enables multiple developers to work on the same project simultaneously while maintaining a complete history of modifications. Git records every change as a commit, allowing users to revisit previous versions, compare changes, and restore files when necessary.

Version control helps developers manage changes efficiently, collaborate without overwriting each other's work, and maintain project integrity. Git provides features such as branching and merging, which allow developers to create separate branches for new features or bug fixes and later combine them into the main project without affecting the stable codebase.

A Git repository is created using the git init command or by cloning an existing repository with git clone. Changes are tracked by adding files to the staging area using git add and saving them permanently with git commit. Commands such as git status, git log, and git diff help users monitor the repository, view commit history, and compare file changes.

Git also supports remote repositories hosted on platforms such as GitHub, GitLab, and Bitbucket. Commands like git push, git pull, and git fetch enable synchronization between local and remote repositories, making collaboration easier and ensuring that all team members have access to the latest project updates.

Performing version control operations using Git improves code management, facilitates teamwork, prevents data loss, and provides a reliable mechanism for maintaining different versions of a project throughout the software development lifecycle.

| No. | Command | What it does |
|---|---|---|
| 1 | git init | Initializes a new Git repository in the current folder. |
| 2 | git clone <repository-url> | Downloads an existing Git repository to your computer. |

| No. | Command | What it does |
|---|---|---|

NAME:- HITEN PAHILAJANI

ROLL NO:- 2401069

BATCH:- T13

SUBJECT:- ASD&D

| | | |
|---|---|---|
| 3 | git status | Shows the current status of files (modified, staged, untracked, etc.). |
| 4 | git add <file> | Stages a specific file for the next commit. |
| 5 | git add . | Stages all modified and new files. |
| 6 | git commit -m "message" | Saves the staged changes with a commit message. |
| 7 | git log | Displays the commit history. |
| 8 | git log --oneline | Displays a shortened commit history. |
| 9 | git diff | Shows changes between the working directory and the staging area. |
| 10 | git diff --staged | Shows changes between the staging area and the last commit. |
| 11 | git branch | Lists all local branches. |
| 12 | git branch <branch-name> | Creates a new branch. |
| 13 | git checkout <branch-name> | Switches to another branch. |
| 14 | git checkout -b <branch-name> | Creates and switches to a new branch. |
| 15 | git switch <branch-name> | Switches to an existing branch (newer alternative to checkout). |
| 16 | git merge <branch-name> | Merges the specified branch into the current branch. |
| 17 | git remote -v | Shows the configured remote repositories. |

NAME:- HITEN PAHILAJANI

ROLL NO:- 2401069

BATCH:- T13

SUBJECT:- ASD&D

| | | |
|---|---|---|
| 18 | git remote add origin <repositoryurl> | Adds a remote repository named origin. |
| 19 | git push origin <branch-name> | Uploads commits to the remote repository. |
| 20 | git pull origin <branch-name> | Downloads and merges changes from the remote repository. |
| 21 | git fetch | Downloads changes from the remote without merging them. |
| 22 | git reset --hard HEAD | Discards all local changes since the last commit. *(Use with caution.)* |
| **No.** | **Command** | **What it does** |
| 23 | git restore <file> | Restores a file to its last committed state. |
| 24 | git rm <file> | Removes a file from both the working directory and Git tracking. |
| 25 | git config --global user.name "Your Name" | Sets your global Git username. |

NAME:- HITEN PAHILAJANI

ROLL NO:- 2401069

BATCH:- T13

SUBJECT:- ASD&D

**Output :**

```
hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git init
Initialized empty Git repository in C:/Users/hiten/OneDrive/Desktop/project/.git/

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git clone https://github.com/hitenpahilajani06/git-project.git
Cloning into 'git-project'...
warning: You appear to have cloned an empty repository.

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        git-project/

nothing added to commit but untracked files present (use "git add" to track)

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ touch file.txt

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git add file.txt

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git add .
error: 'git-project/' does not have a commit checked out
error: unable to index file 'git-project/"
fatal: adding files failed

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git commit -m "commited"
[main (root-commit) fc6f93e] commited
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file.txt

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git log
commit fc6f93ef1bce9549d9fb2e44914855399039 26b6 (HEAD -> main)
Author: hiten pahilajani <hitenpahilajani06@gmail.com>
Date:   Tue Jul 14 18:27:56 2026 +0530

    commited
```

NAME:- HITEN PAHILAJANI

ROLL NO:- 2401069

BATCH:- T13

SUBJECT:- ASD&D

```
hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git log --oneline
fc6f93e (HEAD -> main) committed

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git diff

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ echo "Hello World" > file.txt

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git add file.txt
warning: in the working copy of 'file.txt', LF will be replaced by CRLF the next time Git touches it

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git commit -m "second commit"
[main fb349ed] second commit
 1 file changed, 1 insertion(+)

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git diff

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ echo "git project" > file.txt

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git diff
warning: in the working copy of 'file.txt', LF will be replaced by CRLF the next time Git touches it
diff --git a/file.txt b/file.txt
index 557db03..74788a2 100644
--- a/file.txt
+++ b/file.txt
@@ -1 +1 @@
-Hello World
+git project

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git diff --staged
```

NAME:- HITEN PAHILAJANI

ROLL NO:- 2401069

BATCH:- T13

SUBJECT:- ASD&D

```
hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ echo "Hello world" > file.txt

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git add file.txt
warning: in the working copy of 'file.txt", LF will be replaced by CRLF the next time Git touches it

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git diff --staged

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git status
On branch main
.
Untracked files:
  (use "git add  <file>..." to include in what will be committed)
        file.txt

nothing added to commit but untracked files present (use "git add" to track)

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git add
error: 'git-project/' does not have a commit checked out
error: unable to index file 'git-project/'
fatal: adding files failed

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ ^C
error: 'git-project/' does not have a commit checked out
error: unable to index file 'git-project/'
fatal: adding files failed

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git commit -m "second commit"
On branch main
Untracked files:
  (use "git add  <file>..." to include in what will be committed)
        git-project/

nothing added to commit but untracked files present (use "git add" to track)
hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git branch
* main
```

NAME:- HITEN PAHILAJANI

ROLL NO:- 2401069

BATCH:- T13

SUBJECT:- ASD&D

```
hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git branch branch2

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git checkout branch
error: pathspec 'branch' did not match any file(s) known to git

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git checkout branch2
Switched to branch 'branch2'

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (branch2)
$ git checkout -b branch3
Switched to a new branch 'branch3'

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (branch3)
$ git switch branch
fatal: invalid reference: branch

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (branch3)
$ git switch main
Switched to branch 'main'

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git merge branch2
Already up to date.

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git remote -v

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git remote add origin https://github.com/hitenpahilajani6/git-project.git

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git remote -v
origin  https://github.com/hitenpahilajani06/git-project.git (fetch)
origin  https://github.com/hitenpahilajani06/git-project.git (push)

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git push origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (6/6), 437 bytes | 145.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/hitenpahilajani06/git-project.git
```

NAME:- HITEN PAHILAJANI

ROLL NO:- 2401069

BATCH:- T13

SUBJECT:- ASD&D

```
hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
Compressing objects: 100% (22/2), done.
Writing objects: 100% (6/6), 437 bytes | 145.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/hitenpahilajani06/git-project.git
 * [new branch]       main -> main

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git pull origin main
From https://github.com/hitenpahilajani06/git-project
 * branch            main       -> FETCH_HEAD
Already up to date.

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git fetch

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git restore file.txt

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git rm file.txt
rm 'file.txt'

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git config --get user.name
hiten pahilajani

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git config --get user.email
hitenpahilajani06@gmail.com

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git config --global user.name "hiten pahilajani"

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git config --global user.email "hitenpahilajani06@gmail.com"

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git config --get user.email
hitenpahilajani06@gmail.com

hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git config --get user.name
hiten pahilajani
hiten@LAPTOP-I8SDNHC7 MINGW64 ~/OneDrive/Desktop/project (main)
$ git is now at 7b349ed second commit
```

**Conclusion :**

Through this experiment, I learned how to perform basic version control operations using Git. I understood how to initialize a repository, track file changes, stage and commit updates, view the status and history of a project, create and manage branches, and synchronize a local repository with a remote repository. I also learned how commands such as git reset, git push, and git pull help manage project versions efficiently. This experiment enhanced my understanding of version control and demonstrated how Git simplifies project management, collaboration, and code maintenance.